

Amortized Deduction for Binary Linear Systems: Learning Branching Guidance from Exact Conditional Marginals

neuroMCTS project

September 2026

Abstract

A binary linear system (BLS) asks for $\mathbf{x} \in \{0, 1\}^n$ satisfying $A\mathbf{x} = \mathbf{b}$ with $A \in \{0, 1\}^{m \times n}$, a formulation that arises in laser-based non-destructive testing, where a binary occupancy image is reconstructed from integer projections. On market-split-style instances the linear relaxation is deliberately uninformative, so backtracking search spends most of its effort on branching decisions that a cheap heuristic cannot make well. We present a learned branching heuristic for this setting and an exact procedure for training it.

The method rests on two observations. First, conditioning a BLS on a partial assignment is *identical* to substituting the assignment out and obtaining a smaller BLS; a single network therefore predicts at the root and at every interior node without architectural change, and training states are ordinary instances of reduced size. Second, on instances small enough to enumerate, the exact conditional marginal $p_j = \Pr[x_j = 1 \mid A, \mathbf{b}]$ is computable, and it partitions the free variables of a node into those every surviving solution agrees on — where branching the other way empties the subtree and forces a backtrack — and those solutions disagree on, where either branch still reaches a solution. Only the first kind carries cost. This partition supplies both an exact training target and a criterion for which predictions matter: at the root only 6.3% of variables are of the first kind, but by depth 8 the share is 89.0%, which is where prediction accuracy begins converting into search cost.

We train a 21.8K-parameter bipartite graph network on these targets and use it for branching only, never for fixing variables, since it produces a probability rather than a proof. Against LP-probing — the sound procedure that detects the same variables, at one linear program each — the network needs a single relaxation and one forward pass, so their cost ratio grows linearly in instance size: $1.7\times$ at $n = 25$, $7.2\times$ at $n = 60$, $17.8\times$ at $n = 150$, measured with a warm-started backend on both sides. Under a matched wall-clock budget, replicated over three seeds, the resulting search is $3.29\times$ faster in median than LP-guided branching at 21×60 (sign test $p = 0.019$ per seed), expands $6.2\times$ fewer nodes, and solves 30/30 against 25/30 in every replicate; within the full pipeline, where lattice reduction runs first, guidance closes every residual instance in all three seeds (50/50) against 45/50, lifting the end-to-end rate from $94.4 \pm 5.1\%$ to $100 \pm 0.0\%$. Controlling the number of solutions across sizes, we find no measurable transfer penalty: a network trained only at 10×25 is statistically indistinguishable from one trained at the evaluation size.

We also report what the method does not achieve. CP-SAT solves the same instances 41–62 $\times$ faster at every size tested, and our system does not compete with it; decomposing that gap shows our branching produces $1.6\times$ *fewer* nodes but processes them $78\times$ more slowly, locating the deficit in a research implementation rather than in search quality. And the cost advantage over probing, while asymptotically real, is only 1.3 – $1.7\times$ at the depths where prediction matters most on our instances, where probing additionally offers proofs rather than predictions. The contribution we claim is therefore methodological: exact conditional posteriors are an obtainable and well-posed supervision signal for branching, in-tree states are a better training distribution than root states, and the resulting heuristic transfers across instance size. Code, generators, exact-marginal labels and per-instance results are released.

1 Introduction

1.1 Problem

Let $A \in \{0, 1\}^{m \times n}$ and $\mathbf{b} \in \mathbb{Z}^m$. A *binary linear system* (BLS) is the feasibility problem

$$\text{find } \mathbf{x} \in \{0, 1\}^n \text{ such that } A\mathbf{x} = \mathbf{b}, \quad (1)$$

whose solution set we write $\mathcal{S} = \{\mathbf{x} \in \{0, 1\}^n : A\mathbf{x} = \mathbf{b}\}$. Each row is a cardinality constraint $\sum_j a_{ij}x_j = b_i$ stating how many cells of a designated subset are occupied.

The motivating application is laser-based non-destructive testing. A specimen is scanned along m directions; each scan returns the integer count of absorbing cells along its path, and the task is to reconstruct the binary occupancy image. Two properties of that setting shape the problem formulation. Solutions need not be unique — several images can be consistent with the same measurements, and recovering *any* member of $\mathcal{S}$ solves the physical problem. And measurement noise can perturb $\mathbf{b}$ so that $\mathcal{S} = \emptyset$, making a reliable infeasibility verdict as valuable as a solution.

Deciding $\mathcal{S} \neq \emptyset$ is NP-complete in general. The instances studied here are drawn from a deliberately hard regime modelled on market-split problems (Cornuéjols and Dawande, 1999; Aardal et al., 2000b), in which the linear relaxation

$$\tilde{\mathcal{S}} = \{\mathbf{x} \in [0, 1]^n : A\mathbf{x} = \mathbf{b}\} \quad (2)$$

is a large polytope with predominantly fractional vertices. Relaxation-based bounding is therefore weak, and a complete solver must decide where to branch with little guidance from the relaxation — exactly the decision this paper learns.

1.2 Where a learned component attaches, and what to train it on

A complete solver for (1) alternates deduction (constraint propagation, relaxation-based pruning) with a *branching decision*: which free variable to split on, and which value to try first. Deduction is sound and must remain so; branching is a heuristic choice, where an error costs search time but never correctness. This asymmetry makes branching the natural place to attach a learned component, and it is where learning has been most successful in mixed-integer programming (Khalil et al., 2016; Gasse et al., 2019).

The difficulty is supervision. Two standard answers are available and both are unsatisfactory here. Training against a planted solution $\mathbf{x}^*$ treats one member of $\mathcal{S}$ as ground truth, which is ill-posed when $|\mathcal{S}| > 1$: nothing in $(A, \mathbf{b})$ distinguishes the planted solution from any other, so the label is a sample from the posterior rather than the posterior. Imitating an expensive expert, as in strong-branching imitation, requires an expert whose decisions are both informative and affordable to collect; the natural candidate here, LP-probing, costs one linear program per variable per node.

We take a third route, available because these instances have planted solutions and because feasibility can be enumerated at moderate size: compute the *exact conditional posterior* by exhaustive enumeration on small instances, and train against it. Section 3 shows that this is both tractable and, in a precise sense, the right target — and that the posterior additionally identifies *which* predictions can affect search at all.

1.3 Contributions

1. **A reduction identity that makes in-tree supervision trivial to implement.** Conditioning (1) on a partial assignment yields a smaller BLS with the same structure (Proposition 1).

One architecture, one label generator and one training loop therefore serve the root and every interior node, and a depth- d training state is stored as an ordinary instance on $n - d$ variables.

2. **A partition of branching decisions by whether they can cost anything.** The exact marginals split a node’s free variables into DET (all surviving solutions agree; branching the other way empties the subtree) and OPEN (solutions disagree; either branch reaches a solution). Only DET errors cost a backtrack. We measure the composition: 6.3% DET at the root, rising to 89.0% at depth 8 (Sections 4.3, 4.4). This locates useful prediction inside the tree rather than at the root and explains why root-level accuracy is a poor objective.
3. **A cost characterization of the learned heuristic against the sound procedure it approximates.** The network needs one relaxation and one forward pass; LP-probing needs one relaxation per variable. Measured with a warm-started backend on both sides, the ratio scales from $1.7\times$ at $n = 25$ to $17.8\times$ at $n = 150$ (Section 4.5). We report this as a scaling law rather than a single figure, and note that at the sizes where our search operates the advantage is modest and probing has the additional merit of soundness.
4. **End-to-end evaluation under matched wall-clock budgets.** At 21×60 , replicated over three seeds, the learned guidance is $3.29\times$ faster in median than LP-based branching (sign test $p = 0.019$ per seed), expands $6.2\times$ fewer nodes, and solves 30/30 against 25/30 within 600 s in every replicate (Section 4.6). In the deployed cascade it closes every residual instance in all three seeds, $100 \pm 0.0\%$ against $94.4 \pm 5.1\%$ (Section 4.9).
5. **Size transfer with solution multiplicity controlled.** Holding $|\mathcal{S}|$ comparable across sizes, a network trained only at 10×25 matches one trained at the evaluation size 21×60 (faster on 15/30 instances; total time differing by 0.4%), showing the learned quantity is not size-specific (Section 4.8).

2 Related Work

2.1 Binary linear systems and market-split hardness

Problem (1) is a 0/1 integer feasibility problem and is NP-complete for general A . Practical hardness, however, depends on the instance distribution. Cornuéjols and Dawande (1999) introduced the market-split family, in which m agents must divide n items so that every agent’s budget is met exactly; with $n \approx 10(m - 1)$ these instances defeat branch-and-bound at remarkably small sizes. The mechanism is the integrality gap: the relaxation polytope is large and its vertices are fractional, so bounding prunes little and the search must branch nearly blindly. Aardal et al. (2000b) showed that lattice basis reduction attacks the same family far more effectively than branch-and-bound, a result we rely on in Section 3.2.

Our generator (Section 4.1) makes this mechanism explicit by maximizing the spread of relaxation vertices, which enlarges $\tilde{\mathcal{S}}$ and weakens the relaxation signal by construction.

2.2 Complete solvers

Mixed-integer programming. Branch-and-bound over the linear relaxation, strengthened by presolve and cutting planes, is the standard approach. Its leverage comes from pruning nodes whose relaxation is infeasible or bounded away from the target, which is precisely what the market-split construction removes.

Constraint programming with clause learning. CP-SAT-style solvers combine constraint propagation with conflict-driven clause learning (Marques-Silva and Sakallah, 1999; Moskewicz et al., 2001) and lazy clause generation (Ohrimenko et al., 2009). Their strength on this family comes from a property central to our argument: a search *manufactures* information as it descends. A marginal $\Pr[x_j = 1 \mid A, \mathbf{b}]$ may be uninformative at the root while $\Pr[x_j = 1 \mid A, \mathbf{b}, x_1=1, x_5=0]$ is 0 or 1. Structure invisible at the root becomes deducible after commitments are made, which is why Section 3.4 studies the interior of the tree rather than the root.

Lattice reduction. An alternative attack embeds (1) in a lattice and searches for short vectors. LLL (Lenstra et al., 1982) reduces a basis in polynomial time and BKZ (Schnorr and Euchner, 1994) generalizes it with a block size trading time against basis quality. The Aardal–Hurkens–Lenstra embedding (Aardal et al., 2000a,b) is constructed so that a 0/1 solution appears directly as a short vector, making reduction a certificate-producing procedure rather than a heuristic. We use it as the first stage of the deployed pipeline.

2.3 Learning for combinatorial optimization

Bengio et al. (2021) survey this field and frame it in terms we adopt: solvers rely on hand-crafted heuristics for decisions that are “otherwise too expensive to compute or mathematically not well defined,” and learning is a candidate for making those decisions better. The phrase *too expensive to compute* presumes the decision is computable in principle and asks only about cost, which is the situation we measure in Section 4.5.

Branching imitation. The clearest success in this area is imitation of an expensive expert. Strong branching scores candidates by tentatively solving both children’s relaxations; it yields small trees and is too slow to run at every node. Gasse et al. (2019) train a bipartite graph network to imitate it and recover most of the benefit at a fraction of the cost, following Khalil et al. (2016). The gain is not new information — the expert already knew the answer — but cheap access to it. Our method occupies the same structural position, with two differences: the target is an exact posterior rather than an expert’s scores, and the analysis identifies which of the expert’s decisions matter.

Direct satisfiability prediction. Attempts to have a network decide satisfiability end-to-end have a weaker record. NeuroSAT (Selsam et al., 2019) classifies satisfiability by message passing over a literal–clause graph and succeeds on small random instances, but Li et al. (2023) report substantial accuracy collapse under size transfer. Chen et al. (2023) prove that standard message-passing networks cannot separate certain feasible/infeasible MILP pairs at all, since the pairs are indistinguishable in the Weisfeiler–Lehman hierarchy that bounds such architectures (Xu et al., 2019). We therefore do not ask the network for a feasibility verdict; it supplies branching guidance inside a complete search that remains responsible for soundness.

Graph representations. We use graph attention (Veličković et al., 2018) over the bipartite variable–constraint graph, the standard encoding for MILP in this literature (Gasse et al., 2019).

2.4 Amortized optimization

The distinction organizing our results predates its use in combinatorial optimization. Gershman and Goodman (2014) introduced *amortized inference*: when many related queries must be answered,

solving each from scratch is wasteful and the question becomes how to reuse computation across queries. Amos (2023) develops the same idea as *amortized optimization* — using learning to predict solutions by exploiting shared structure across similar instances — and surveys its appearance in variational inference, meta-learning and control. Millidge (2022) states the trade-off in the form most relevant here: *direct* optimization spends computation on the instance at hand and pays that cost at every invocation, whereas *amortized* optimization converts the problem into supervised learning with cheap inference but requires a corpus of solved instances up front and is bounded by the approximator’s generalization; the productive arrangement is hybrid, with amortized predictions steering a direct optimizer.

Our system instantiates that hybrid precisely. The direct method is LP-probing, sound and costly. The amortized method is the network, cheap and unsound. The hybrid rule is that the network steers branching while probing and propagation retain exclusive authority over variable fixing (Section 3.7).

3 Proposed Method

3.1 Setting and notation

For a partial assignment fixing variables $F \subseteq \{1, \dots, n\}$ to values $\mathbf{v} \in \{0, 1\}^{|F|}$, write

$$\mathcal{S}(F, \mathbf{v}) = \{\mathbf{x} \in \mathcal{S} : \mathbf{x}_F = \mathbf{v}\} \quad (3)$$

for the solutions consistent with it, and let $K = \{1, \dots, n\} \setminus F$ index the free variables. The *conditional marginal* of a free variable $j \in K$ is

$$p_j(F, \mathbf{v}) = \frac{|\{\mathbf{x} \in \mathcal{S}(F, \mathbf{v}) : x_j = 1\}|}{|\mathcal{S}(F, \mathbf{v})|}. \quad (4)$$

3.2 System overview

The deployed solver is a cascade of a lattice stage and a guided complete search:

$$\text{propagation} \rightarrow \text{LP infeasibility rule} \rightarrow \text{kernel pump} \rightarrow \text{AHL/BKZ} \rightarrow \text{guided backtracking search}. \quad (5)$$

The first three stages are cheap and sound. AHL/BKZ attempts a certificate-producing lattice reduction with a size-dependent block parameter; when it succeeds it returns a verified solution outright. The learned component appears only in the final stage, and only in its branching decisions. This placement matters for the evaluation: because the lattice stage is independent of the branching heuristic, every heuristic under comparison receives exactly the same residual set of instances, so conditioning on that residual describes the algorithm rather than selecting favourable cases.

3.3 Conditioning is reduction

Proposition 1. *Let A_F and A_K denote the column submatrices of A indexed by F and K . Then*

$$\mathcal{S}(F, \mathbf{v}) \cong \{\mathbf{y} \in \{0, 1\}^{|K|} : A_K \mathbf{y} = \mathbf{b} - A_F \mathbf{v}\}, \quad (6)$$

the bijection being $\mathbf{x} \mapsto \mathbf{x}_K$. Consequently the conditional marginals (4) of the original instance are the ordinary marginals of the reduced instance $(A_K, \mathbf{b} - A_F \mathbf{v})$.

Proof. $A\mathbf{x} = A_F\mathbf{x}_F + A_K\mathbf{x}_K$. Fixing $\mathbf{x}_F = \mathbf{v}$ gives $A\mathbf{x} = \mathbf{b} \iff A_K\mathbf{x}_K = \mathbf{b} - A_F\mathbf{v}$, and $\mathbf{x} \mapsto \mathbf{x}_K$ is a bijection between the two sets since $\mathbf{x}_F$ is determined. The marginal statement follows by counting. $\square$

The identity is elementary but it is what makes in-tree supervision practical. A depth- d search node is not a special object requiring its own encoding; it is an instance of the same problem on $n - d$ variables. One network, one feature extractor and one label generator therefore cover the whole tree, and training data at depth d is produced by substitution rather than by instrumenting a solver.

Two implementation details preserve correctness. Rows that become all-zero after substitution carry no constraint and are removed; retaining them causes linear-programming backends to reject the model. And the free bound condition $0 \leq \mathbf{b} - A_F\mathbf{v} \leq A_K\mathbf{1}$ is checked before any relaxation is solved, since it detects a large fraction of dead nodes at negligible cost.

3.4 Which predictions can cost anything

At a node with surviving solution set $\mathcal{S}(F, \mathbf{v}) \neq \emptyset$, partition the free variables by their conditional marginal:

$$\text{DET} = \{j \in K : p_j \in \{0, 1\}\}, \quad \text{OPEN} = \{j \in K : 0 < p_j < 1\}. \quad (7)$$

The distinction is operational rather than statistical:

- For $j \in \text{OPEN}$, both children of a branch on x_j contain solutions. Either choice keeps a solution reachable, so *no branching decision on a OPEN variable can cost anything*.
- For $j \in \text{DET}$, one child is empty. Branching against the forced value guarantees that the subtree is exhausted without a solution, costing a full backtrack.

This gives the conditional marginal its operational reading. It is not a “probability of being correct” about some designated solution, but the *fraction of the surviving solution set retained* by an assignment; and the only errors that matter are those on variables where that fraction is 0 or 1.

Two consequences follow, both of which the experiments confirm. First, an aggregate accuracy over all free variables is a poor objective, because it mixes decisions that can cost a backtrack with decisions that cannot; the relevant quantity is accuracy on DET. Second, since $|\mathcal{S}(F, \mathbf{v})|$ shrinks as the search descends, the DET share grows with depth, and useful prediction is concentrated in the interior of the tree rather than at the root.

3.5 Exact conditional marginals as training targets

For instances small enough to enumerate, $\mathcal{S}$ is obtained exactly with a constraint solver in all-solutions mode, and (4) is computed by counting. Training minimizes the cross-entropy to the resulting Bernoulli targets,

$$\mathcal{L} = -\frac{1}{|K|} \sum_{j \in K} \left[p_j \log \hat{p}_j + (1 - p_j) \log(1 - \hat{p}_j) \right], \quad (8)$$

which is minimized exactly at $\hat{p}_j = p_j$. Note that the same loss with a planted solution in place of p_j — the conventional choice — is a noisy surrogate whenever $|\mathcal{S}| > 1$, since the planted solution is one draw from the posterior; on our instances a Bayes-optimal predictor agrees with it only about two thirds of the time.

Enumeration correctness. A subtlety here is easy to get wrong and silently corrupts every label derived from it. Constraint solvers report distinct statuses for “search exhausted” and “time limit reached with solutions already collected.” Only the former certifies that $\mathcal{S}$ is complete; accepting the latter substitutes a truncated solution set, which biases every marginal computed from it. In our implementation only **OPTIMAL** and **INFEASIBLE** are accepted and timed-out instances are discarded rather than averaged in. A useful diagnostic is that mean enumeration time equal to the time limit indicates truncated instances being counted as complete.

Sampling reachable states. A depth- d training state is built by drawing d from a uniform range, taking a prefix of an LP-confidence variable ordering, and fixing those variables to the values of a randomly chosen *actual solution*. Fixing arbitrary values would produce states with $\mathcal{S}(F, \mathbf{v}) = \emptyset$; such states are dead nodes that propagation prunes immediately and on which the conditional marginal is undefined, so they are not states a branching heuristic is ever asked about.

3.6 Network

An instance is encoded as a bipartite graph with n variable nodes, m constraint nodes, and an edge (j, i) whenever $a_{ij} = 1$. Let $\tilde{\mathbf{x}}$ be the relaxation solution of the current (reduced) instance and $r_i = \sum_j a_{ij}$ the row sums. Node features are

$$\text{variable } j : (\tilde{x}_j, \deg(j)/m, |\tilde{x}_j - \text{round}(\tilde{x}_j)|), \quad (9)$$

$$\text{constraint } i : (b_i/r_i, r_i/n, (b_i - A_i\tilde{\mathbf{x}})/r_i), \quad (10)$$

i.e. relaxation value, column degree and fractionality for variables; tightness, scaled row size and relaxation residual for constraints. All are size-normalized, which is what permits a single parameter set to apply at any (m, n) .

Both node types are embedded to $h = 64$ dimensions by a linear map and refined by $L = 4$ rounds of bidirectional message passing with graph attention (Veličković et al., 2018):

$$\mathbf{h}^{\text{con}} \leftarrow \text{LN}(\text{ELU}(\text{GAT}(\mathbf{h}^{\text{var}} \rightarrow \mathbf{h}^{\text{con}})) + \mathbf{h}^{\text{con}}), \quad (11)$$

$$\mathbf{h}^{\text{var}} \leftarrow \text{LN}(\text{ELU}(\text{GAT}(\mathbf{h}^{\text{con}} \rightarrow \mathbf{h}^{\text{var}})) + \mathbf{h}^{\text{var}}), \quad (12)$$

followed by a two-layer head emitting one logit per variable, $\hat{p}_j = \sigma(z_j)$. The model has 21,761 parameters.

The architecture is deliberately small and single-headed. Its only output is the quantity the objective (8) defines; there are no auxiliary value or feasibility heads, because the search draws soundness from its symbolic components and asks the network for nothing else.

3.7 Inference: guidance, not commitment

The network is consulted once per node. Branching selects

$$j^* = \arg \max_{j \in K} |\hat{p}_j - \tfrac{1}{2}|, \quad \text{first value} = \mathbb{K}[\hat{p}_{j^*} \geq \tfrac{1}{2}], \quad (13)$$

that is, the variable the network is most confident about, tried at its predicted value. The search remains complete and a wrong prediction costs backtracking only.

Variables are *fixed* only by sound deduction. Two mechanisms are available: constraint propagation, and LP-probing, which fixes x_j when assigning the opposite value makes even the relaxation infeasible. The division of labour follows directly from the measurements in Section 4.5: where a

wrong answer costs time, the cheap unsound predictor is preferable; where a wrong answer costs correctness, the expensive sound procedure is required and cannot be replaced.

Algorithm 1 states the resulting search.

Algorithm 1 Guided backtracking search for a BLS

Input: instance $(A, \mathbf{b})$, guidance model f_θ , time budget T
stack $\leftarrow [(A, \mathbf{b})]$
while stack $\neq \emptyset$ **and** elapsed $< T$:
 $(A', \mathbf{b}') \leftarrow \text{pop}(\text{stack})$
 if $\mathbf{b}' < \mathbf{0}$ **or** $\mathbf{b}' > A'\mathbf{1}$: **continue** // *free bound check*
 drop all-zero rows of A' ; **if** none remain: **return** any completion
 propagate; **if** contradiction: **continue**
 if propagation determined a set D of variables:
 push reduce($A', \mathbf{b}', D$); **continue** // *Proposition 1*
 if all variables assigned: **return** solution if verified, else **continue**
 $\hat{\mathbf{p}} \leftarrow f_\theta(A', \mathbf{b}')$ // *one forward pass, all variables*
 $j^* \leftarrow \arg \max_j |\hat{p}_j - 1/2|$; $v \leftarrow \mathbb{I}[\hat{p}_{j^*} \geq 1/2]$
 push reduce($A', \mathbf{b}', \{j^* = 1-v\}$), then reduce($A', \mathbf{b}', \{j^* = v\}$)
return unresolved

3.8 Cost model

The comparison that motivates the design is per-node cost for a whole-instance verdict, and its structure is asymptotic rather than a constant factor. Let n_c be the number of free variables at a node. The network solves *one* relaxation to build its features and performs *one* forward pass, so its LP count is $O(1)$ in n_c . LP-probing solves one relaxation per candidate variable, $O(n_c)$. The ratio between them therefore grows linearly in instance size, and any single number quoted for it is meaningful only together with the n_c at which it was measured.

This also means the comparison is sensitive to how the LP backend is implemented, in a way that cuts against the learned method. A backend that rebuilds the model for every call inflates probing’s cost by its full $O(n_c)$ factor while charging the network only once, exaggerating the advantage. A correctly engineered branch-and-bound keeps one relaxation and re-optimizes from the parent basis after a bound change, which is an order of magnitude cheaper per probe. Section 4.5 measures both methods against a warm-started backend for this reason, and reports the ratio as a function of n_c rather than as a single figure.

4 Experiments

4.1 Instances and control of solution multiplicity

Instances are generated by the procedure GENHARD(m, n, K): draw a random 0/1 matrix A of density 1/2; plant $K = 20$ candidate solutions $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(K)}$ with $\mathbf{b}^{(k)} = A\mathbf{x}^{(k)}$; retain the one maximizing *vertex spread*, the mean pairwise L_2 distance among relaxation vertices obtained from random objective directions. Maximizing vertex spread enlarges $\tilde{\mathcal{S}}$ directly and is the mechanism that makes market-split instances hard for relaxation-based methods (Section 2). Uniqueness of the planted solution is not enforced.

Why $|\mathcal{S}|$ must be controlled across sizes. Solution multiplicity changes the *character* of the prediction task, not only its difficulty. When $|\mathcal{S}| = 1$ every free variable is DET and predicting well is tantamount to solving; when $|\mathcal{S}|$ is large most variables are OPEN and the task is to estimate a diffuse posterior. Comparing sizes without controlling $|\mathcal{S}|$ therefore confounds size with task identity. Since $|\mathcal{S}|$ falls as m rises at fixed n , we select m per size to match it.

Size	m	m/n	median $ \mathcal{S} $	root ceiling	conditional ceiling
10×25	10	0.40	23	70.4%	87.0%
18×50	18	0.36	19	71.3%	87.2%
21×60	21	0.35	10	—	—

Table 1: Sizes used, with solution multiplicity matched. Ceilings (the Bayes-optimal per-variable accuracies of Section 4.3) agree to within one point between the two training sizes, so size is the variable that changes.

Limits of the control. Matching $|\mathcal{S}|$ forces m/n to vary ($0.40 \rightarrow 0.35$); at fixed n the two cannot be held simultaneously and we prioritize $|\mathcal{S}|$ because it determines task identity. The control is also granular: at $n = 60$ the median $|\mathcal{S}|$ steps $42 \rightarrow 12 \rightarrow 2$ across $m = 20, 21, 22$, so $m = 21$ is the best available integer and the realized evaluation set has median $|\mathcal{S}| = 10$ with 27/30 verified.

Where enumeration stops. At $n = 70$ no enumeration completed within 120s at the m giving the target multiplicity (0/10). At $n = 100$ the obstruction is more basic: a constraint solver finds *zero* solutions within 20s at $m \in \{28, 32, 36, 40\}$ despite a planted one. At that size the bottleneck is finding any solution rather than choosing branches well, so a branching comparison there would not be informative. We therefore evaluate at $n \leq 60$ and report this boundary as a limitation.

4.2 Implementation and protocol

Software. Relaxations are solved with SCIP via PySCIPOpt; enumeration and the CP-SAT baseline use OR-Tools; lattice reduction uses fpylll (LLL and BKZ). The network is implemented in PyTorch with PyTorch Geometric (GATConv).

Training. Adam, learning rate 10^{-3} , 20 epochs over 10,000 in-tree states, batch size 1 (instances vary in size), gradient-norm clipping at 1.0, loss (8) with exact marginal targets. Training states are drawn at depths 0–12 uniformly, 6 states per source instance, from a pool of 1,760 instances; evaluation uses 440 instances from a disjoint pool. Training takes approximately 20 minutes on one CPU core.

Threading. All timing-sensitive components run single-threaded (`torch.set_num_threads(1)`). On graphs of this size multi-threaded BLAS is counterproductive: we measure 11.3ms per forward pass at 8 threads against 0.9ms at one.

Search evaluation. Each arm is run on identical instances with a hard per-instance wall-clock budget enforced by subprocess termination, since lattice reduction executes inside a blocking native call that in-process timers cannot interrupt. Instances run at concurrency 6 on a 20-core machine. This low concurrency is deliberate: wall-clock-budgeted comparisons are sensitive to CPU contention,

and at higher concurrency we have observed instances solvable in 14–15 s standalone exceeding a 20 s cap.

Statistics. Solve-rate differences between arms on the same instances are tested with McNemar’s exact test; per-instance speed differences with a two-sided sign test. We report both rather than a single aggregate because they can have different power. The main comparison at 21×60 is replicated over three independent seeds and we report the spread; smaller-size and transfer results are single-run and marked as such. Seeds are set explicitly: an earlier version derived them from `hash()` of the instance filename, which Python randomizes per process, so those runs had no recorded provenance.

4.3 Prediction quality against the Bayes ceiling

Because $\mathcal{S}$ is enumerated exactly, the Bayes-optimal per-variable accuracy is computable:

$$\text{ceil} = \frac{1}{|K|} \sum_{j \in K} \max(p_j, 1 - p_j), \quad (14)$$

attained by answering the majority value of the surviving solutions. No deterministic predictor of $\mathbf{x}^*$ from $(A, \mathbf{b})$ can exceed it. Table 2 evaluates predictors against it on 400 held-out 10×25 instances with $\mathcal{S}$ fully enumerated.

Predictor	all variables	top-3	top-5	L_1 to true marginal
Bayes ceiling	69.9%	93.1%	90.5%	0.0000
Proposed network	68.6%	87.4%	85.2%	0.1261
trained on soft targets (8)	67.7%	87.2%	85.3%	0.0977
Multi-head GNN (2.11M params)	60.7%	72.0%	69.5%	0.1813
LP relaxation rounding	60.6%	64.5%	64.1%	0.3213

Table 2: Root-level prediction against the exact posterior, 400 held-out 10×25 instances. “top- k ” restricts to the k variables the predictor is most confident about. The multi-head model is a 2.11M-parameter network with selection, assignment, value and feasibility heads; the proposed network has 21.8K parameters.

Two observations. The 21.8K-parameter single-head network comes within 1.9 points of the ceiling and within 5.7 on the top-3 subset, whereas a $97\times$ larger multi-head network is statistically indistinguishable from plain relaxation rounding (60.7% vs. 60.6%); capacity is not the binding constraint, and a model whose output is the target quantity substantially outperforms one that produces it as one of several heads. Training on exact marginals rather than a planted solution improves posterior estimation markedly (L_1 0.1261 $\rightarrow$ 0.0977) while leaving argmax accuracy unchanged, which matters because the marginal, not the argmax, is what the analysis of Section 3.4 consumes.

What the aggregate conceals. Table 3 decomposes the same root variables by the partition of Section 3.4.

At the root, 93.7% of variables are OPEN. The aggregate ceiling of 69.9% is therefore dominated by predictions that cannot affect the search, and on the 6.3% that can, the information is already complete (100%). Root-level accuracy is consequently a weak objective for a branching heuristic, and the operative question is not whether the information exists but what detects it — taken up next.

Root variables	count	share	ceiling on subset
DET (branching against it empties the subtree)	38	6.3%	100.0%
OPEN (either branch reaches a solution)	562	93.7%	67.2%
all variables	600	100%	69.3%

Table 3: Root-level decomposition, 24 instances of 10×25 with $\mathcal{S}$ enumerated. The aggregate is 94% composed of variables on which no branching decision can cost a backtrack.

4.4 Depth, and where the residual lies

Using Proposition 1 we measure, at each depth along a prefix drawn from an actual solution, the conditional ceiling and the fraction of remaining variables that propagation forces (Table 4).

depth	surviving $ \mathcal{S} $	conditional ceiling	forced by propagation	proposed network
0	25.2	70.5%	0.0%	67.9%
4	4.1	82.2%	0.4%	73.2%
7	1.9	92.6%	4.1%	80.4%
8	1.6	93.4%	8.2%	85.0%
12	1.1	98.6%	56.7%	97.4%
15	1.0	99.7%	94.6%	—

Table 4: Conditional information against depth, $n = 25$, 150 instances, LP-confidence variable ordering.

The consequential movement is in composition rather than level: the DET share rises from 6.3% at the root to 89.0% at depth 8. At the root almost every branching decision is inconsequential because both children contain solutions; by depth 8 almost every decision empties a subtree when wrong. Training on in-tree states rather than root states improves accuracy by 5.3–11.5 points over depths 6–10.

Table 5 decomposes the residual gap at these depths and asks what each method detects.

depth	% DET	network on DET	propagation	LP integrality	LP-probing	network on OPEN
5	61.5%	90.4%	0.9%	66.5%	62.2%	57.9%
6	72.1%	89.2%	1.6%	67.3%	68.1%	52.3%
7	83.0%	87.9%	4.1%	71.0%	75.0%	52.9%
8	89.0%	89.2%	7.5%	76.9%	79.4%	59.7%

Table 5: Residual decomposition. Columns 4–6 are recall on the ground-truth DET set; columns 3 and 7 are the network’s argmax accuracy on each subset.

The gap lies entirely on DET variables: at depth 7, $0.83 \times 12.1 \approx 10$ points, matching the measured 12.2-point shortfall against the conditional ceiling. Performance near 50% on OPEN variables is correct behaviour rather than a defect, since those variables are genuinely undetermined. The striking figure is that unit propagation detects only 0.9–7.5% of logically determined variables: the information is present and derivable but not reachable by the cheapest sound rule. LP-probing recovers 62–79%, and together with LP integrality 82.8–92.0%.

4.5 Cost per node

Both methods are charged everything they must pay to reach a whole-instance verdict: the network one relaxation for its features plus one forward pass, probing one relaxation per free variable. Both use the same warm-started backend, a single model whose variable bounds are changed in place and re-optimized from the parent basis (Gurobi dual simplex), which is how a branch-and-bound implementation reuses its relaxation. Timings are single-threaded.

Instances	n	network	LP-probing	ratio	probing recall
10×25	25	3.22 ms	5.60 ms	$1.7\times$	—
18×50	50	2.33 ms	11.84 ms	$5.1\times$	—
21×60	60	2.57 ms	18.39 ms	$7.2\times$	—
40×100	100	4.02 ms	62.46 ms	$15.5\times$	—
60×150	150	4.45 ms	79.06 ms	$17.8\times$	—
<i>in-tree states of 10×25 instances, by depth:</i>					
depth 5	20	2.44 ms	4.02 ms	$1.6\times$	51.6%
depth 6	19	2.39 ms	3.98 ms	$1.7\times$	68.0%
depth 7	18	2.35 ms	3.43 ms	$1.5\times$	72.8%
depth 8	17	2.34 ms	3.13 ms	$1.3\times$	79.8%

Table 6: Per-node cost for a whole-instance verdict, warm-started LP backend for both methods, single-threaded. The network’s accuracy on DET variables at depths 5–8 is 85.9–92.7% against probing’s recall in the last column.

The ratio behaves as the cost model predicts: it is negligible at $n = 25$ and reaches $17.8\times$ at $n = 150$, tracking the $O(n_c)$ versus $O(1)$ difference in relaxation count. Two consequences follow, and they point in opposite directions.

The advantage is real but asymptotic, and small where we can search. At the sizes where our search actually operates ($n = 25$ – 60) the ratio is 1.3 – $7.2\times$. At the depths where prediction matters most (5–8 of a 10×25 instance, $n_c = 17$ – 20) it is 1.3 – $1.7\times$. A practitioner choosing between the two at that scale should prefer probing, which is *sound*: where its test fires the conclusion is a proof, whereas the network never offers a guarantee. The learned alternative becomes compelling only at $n \gtrsim 100$, and Section 4.1 shows that search itself fails there for every strategy we tried.

The measurement is sensitive to baseline engineering. An earlier version of this comparison used a backend that rebuilt the LP model on every call and, additionally, excluded the network’s own feature relaxation from its timer. That combination reported 19 – $23\times$ at depths 5–8, where the correctly measured value is 1.3 – $1.7\times$ — an overstatement of more than an order of magnitude, arising entirely from how the two sides were instrumented rather than from what they compute. We report the corrected figures and note the general hazard: in any cost comparison against a classical procedure, the classical side must be implemented as a practitioner would implement it, and the learned side must be charged for its own preprocessing.

4.6 Search performance

We compare four branching strategies under matched wall-clock budgets with the lattice stage disabled, isolating branching quality. RANDOM selects an arbitrary variable and value; LP selects

the most integral relaxation value and tries that value first; LP-PROBE probes every free variable, soundly fixes those it proves forced, then branches as LP; MODEL uses the proposed network.

Size (budget)	strategy	solve rate	median nodes	median time	nodes/s
10×25 (60 s)	RANDOM	100%	1,514	0.07 s	—
	LP	100%	66	0.12 s	—
	LP-PROBE	100%	12	0.79 s	—
	MODEL	100%	22	0.08 s	—
20×50 (300 s)	RANDOM	0%	—	—	—
	LP	100%	8,876	24.93 s	376
	LP-PROBE	100%	173	28.14 s	6
	MODEL	100%	1,676	6.37 s	273
21×60 (600 s)	LP	$83.3 \pm 0.0\%$	69,015	207.4 ± 5.9 s	172
	MODEL	$100 \pm 0.0\%$	12,010	51.7 ± 2.1 s	135

Table 7: Branching strategy comparison, lattice stage disabled, 30 instances per cell. The 21×60 row is mean $\pm$ standard deviation over three seeds; the smaller sizes are single-run.

Three points. First, node count and wall-clock rank these methods in *opposite* orders: LP-PROBE produces by far the smallest trees ($51\times$ fewer nodes than LP at 20×50) yet is the slowest, processing 6 nodes/s against 376. Evaluating branching heuristics by tree size alone — a common convention — would select the worst method here. Second, MODEL occupies the productive point, cutting nodes $5.3\times$ relative to LP at a per-node cost below LP’s, for $3.9\times$ lower median time. Third, at 21×60 the advantage becomes qualitative, and it replicates. Across three seeds the learned guidance solves 30/30 every time while relaxation-based branching solves 25/30 every time; on the instances both close, the former is faster on 18/25 with a median ratio of $3.29\times$ (seed-wise 3.35, 3.30, 3.24; sign test $p = 0.019$ within each seed). Seed-to-seed variance is negligible — the solve-rate standard deviation is exactly zero and the speedup varies by ± 0.06 — which tells us the tie-breaking randomness is not what the result rests on.

The five instances that separate the arms are the same five in every seed, and no instance is ever closed by LP and missed by MODEL. That consistency is the strongest form this evidence takes, but it does not make the difference significant by the usual test: a 5–0 split on 30 instances gives McNemar $p = 0.0625$, and no amount of replication improves it, since the limiting quantity is the number of instances rather than the number of runs. Establishing the solve-rate advantage at conventional levels would require a larger evaluation set, not more seeds. The speed advantage, tested per instance, is significant within each seed.

4.7 Comparison against complete solvers

Table 7 compares branching strategies inside our own search loop, which measures branching quality but says nothing about how the system stands against an off-the-shelf solver. That comparison is reported here, on the same instances, single-threaded, under the same budget.

The gap is large and we state it plainly: *the contribution of this paper is not a solver that competes with CP-SAT*. It is worth asking where the gap comes from, because the answer bears on what the branching heuristic is actually doing. Table 9 decomposes it at 21×60 into tree size and per-node throughput, using CP-SAT’s own branch counter.

The learned guidance produces a *smaller* search tree than CP-SAT — $1.6\times$ fewer nodes despite

Size	CP-SAT	SCIP	proposed (MODEL)	CP-SAT advantage
10×25	0.002 s	0.009 s	0.08 s	$40\times$
18×50	0.046 s	0.385 s	2.84 s	$62\times$
21×60	1.159 s	3.687 s	47.28 s	$41\times$

Table 8: Median solve time; all methods solve 30/30 at every size. The proposed system does not beat a complete solver at any size tested, and is outperformed even by SCIP, which it already uses internally for relaxations.

	CP-SAT	proposed	ratio
nodes (median)	18,827 branches	12,010	ours $1.6\times$ fewer
throughput	19,782 nodes/s	254 nodes/s	CP-SAT $78\times$ faster
total time (median)	1.065 s	47.28 s	$41\times$

Table 9: Decomposition of the 21×60 gap. CP-SAT additionally learns a median of 5,514 conflict clauses per instance.

CP-SAT’s clause learning — and loses entirely on per-node cost. That cost is an artifact of our implementation rather than of the method: our loop is Python, it rebuilds and re-solves the node relaxation from scratch rather than warm starting from the parent basis, and it materializes a reduced instance at every node instead of maintaining incremental data structures. Section 4.5 measures the same effect directly: switching to a warm-started backend cuts our per-node LP cost several-fold. We did not port the search loop itself, so the throughput column reflects research code, not a limit of the approach.

Two honest caveats attach to this reading. Tree sizes are not perfectly commensurable — CP-SAT’s “branches” include restarts and conflict-driven jumps, and each of its nodes performs more propagation work than ours — so the $1.6\times$ should be read as an order-of-magnitude statement. And a competitive implementation would need more than a language port: incremental relaxations, clause learning, and restarts are all absent from our search, and the last two are what let CP-SAT scale past the sizes where we stop.

4.8 Size transfer

Because the features are size-normalized and the parameters do not depend on (m, n) , one network applies at any size. We test whether that transfers in practice, with $|\mathcal{S}|$ matched as in Table 1.

Training size	Evaluation size	solve rate	median time	median nodes
10×25	10×25	100%	0.08 s	23
10×25	18×50	100%	2.84 s	833
18×50	18×50	100%	3.42 s	1,001
10×25	21×60	100%	47.28 s	12,010
18×50	21×60	100%	54.75 s	14,108

Table 10: Transfer, lattice stage disabled, 30 instances per row. Rows 2–3 and 4–5 each hold the evaluation size fixed and vary only the training size.

At the 21×60 evaluation size the network trained only at 10×25 — a $2.4\times$ smaller problem —

and the one trained at 18×50 are statistically indistinguishable: the former is faster on exactly 15/30 instances, median ratio $0.92\times$, and total times differ by 0.4% (2,924.9 s vs. 2,914.5 s), with both solving 30/30. Root-level prediction agrees: the 10×25 network scores 67.8%, 69.5% and 69.4% at 10×25 , 20×50 and 40×100 , holding its margin over relaxation rounding (+11.0, +7.2, +4.7 points) at every size.

This matters practically because label generation is the binding constraint. Exact marginals require enumeration, which fails beyond $n \approx 60$; transfer means a network trained where labels are obtainable can be deployed where they are not.

4.9 Deployment in the full pipeline

Table 7 isolates branching by disabling lattice reduction. In deployment the lattice stage runs first and the search sees only its residual. Because that stage is independent of the branching heuristic, every arm receives the identical residual set.

Size	strategy	closed by lattice	residual solved by search	end-to-end
10×25	both	30/30	—	100%
18×50	LP / MODEL	24/30	6/6 / 6/6	100% / 100%
21×60	LP	10–16/30	45/50	$94.4 \pm 5.1\%$
21×60	MODEL	10–16/30	50/50	$100 \pm 0.0\%$

Table 11: Cascade results, 600s budget at 21×60 , which is reported over three seeds; smaller sizes are single-run. Lattice coverage varies across seeds (16, 14, 10 of 30) because the lattice stage draws random column permutations, so the residual handed to the search varies with it.

Lattice coverage falls with size — 30/30, 24/30, 16/30 — and that decline is what creates room for branching quality to matter. At 10×25 and 18×50 the residual is empty or trivially closed by either strategy. At 21×60 between ten and twenty instances reach the search depending on the seed, and the learned guidance closes every one of them in all three replicates (50/50 across seeds) while relaxation-based branching closes 45/50. End to end this is $100 \pm 0.0\%$ against $94.4 \pm 5.1\%$. The variance in the LP arm comes from the lattice stage rather than from the branching: when lattice coverage happens to be low, more instances fall through to a search that sometimes fails to close them.

4.10 Does the graph structure contribute?

MarginalNet consumes six scalars per node and propagates them over the bipartite graph. With a feature set that small, a per-variable model ignoring the graph is a serious baseline: if it matches, the message passing is decoration. Table 12 compares models trained on identical targets, loss and splits, differing only in whether structure is used. The aggregated variant appends mean and max of the constraint-side features over each variable’s incident rows, which is the information one message-passing round would deliver.

Message passing contributes: +4.7 points over the best structure-free model and a 36% reduction in L_1 , closing roughly 60% of the residual gap to the ceiling that the MLP leaves open. The improvement in L_1 is the more relevant of the two, since the analysis of Section 3.4 consumes the marginal rather than its argmax.

The same table bounds the claim, though. A four-parameter linear model on the relaxation value alone reaches 78.1%, within 9.2 points of the ceiling; most of the available signal is in that

Model	accuracy	L_1 to marginal	parameters
Linear, variable features only	78.1%	0.2530	4
MLP, variable features only	77.8%	0.2522	8,641
MLP + constraint aggregates	79.4%	0.2361	9,025
MarginalNet (bipartite graph)	84.1%	0.1514	21,761
Bayes ceiling	87.3%	0.0000	—

Table 12: Ablation on in-tree states of 10×25 instances, 2,000 training and 800 test states.

single feature, and the graph adds a useful but not dominant increment on top of it.

4.11 Limitations

Label generation bounds the method, not only its evaluation. Exact marginals require enumerating $\mathcal{S}$, which does not complete beyond $n \approx 60$ on this family. Transfer permits deployment at larger sizes but not training there; approximating p_j by solution sampling is the natural extension and is untested.

The useful size window is bounded on both sides. Below it, lattice reduction closes everything and branching quality is irrelevant. Above it ($n = 100$), search fails outright for every strategy, since a constraint solver finds no solution within 20 s. Our demonstration lives at $n = 60$ and we do not show the window extends further.

Statistical power is limited by instance count, not run count. The solve-rate advantage at 21×60 is 5–0 and identical in all three seeds, but a 5–0 split on 30 instances gives McNemar $p = 0.0625$ at best. Replication cannot improve this; only a larger evaluation set can. The per-instance speed advantage is significant ($p = 0.019$ within each seed).

Multiplicity control is approximate. Median $|\mathcal{S}|$ is 23, 19 and 10 across the three sizes; at $n = 60$ multiplicity steps $42 \rightarrow 12 \rightarrow 2$ across consecutive integer m , so finer matching is unavailable. Three of thirty evaluation instances have unverified $|\mathcal{S}|$ and are marked as such. Matching $|\mathcal{S}|$ also moves m/n from 0.40 to 0.35.

Cost advantage depends on scale and on baseline engineering. The amortization argument holds asymptotically but is worth little at the sizes where our search operates: $1.3\text{--}1.7\times$ at the depths where prediction matters, against a warm-started probing baseline that is additionally sound. It reaches $15\text{--}18\times$ only at $n \geq 100$, where search fails for every strategy we tested. We therefore cannot demonstrate the regime in which the argument is strongest.

The search implementation is not competitive. At 21×60 CP-SAT is $41\times$ faster end to end. Our branching produces fewer nodes, so the deficit is per-node throughput ($78\times$), but we have not shown that a competitive implementation is achievable: incremental relaxations, clause learning and restarts are all absent, and the latter two are load-bearing for CP-SAT at larger sizes.

Single instance family. All results concern one generator from one application. Whether the amortization argument transfers to other constraint families is untested.

5 Conclusion

We presented a learned branching heuristic for binary linear systems together with an exact procedure for supervising it. Two elements carry the method. Conditioning a BLS on a partial assignment is identical to reducing it, so a single network trained on ordinary instances predicts at every node of the search tree without architectural change. And on instances small enough to enumerate, the exact conditional posterior both supplies the training target and identifies which predictions can affect the search at all: only variables that every surviving solution agrees on can cost a backtrack, and their share rises from 6.3% at the root to 89.0% at depth 8.

Measured inside our search loop the heuristic does what it is designed to do. It is $3.29\times$ faster in median than relaxation-based branching at 21×60 , expands $6.2\times$ fewer nodes, and solves 30/30 against 25/30 in each of three seeds; with solution multiplicity controlled, a network trained at $2.4\times$ smaller scale performs indistinguishably from one trained at the evaluation size, which matters because exact supervision is available only at small sizes.

We are equally explicit about what the results do not establish. The system does not compete with a complete solver: CP-SAT is $41\text{--}62\times$ faster at every size we tested. Decomposing that gap is informative rather than exculpatory — our branching yields $1.6\times$ fewer nodes and loses on throughput by $78\times$, which places the deficit in a Python research implementation lacking incremental relaxations, clause learning and restarts — but we have not shown that closing it is achievable. Nor does the cost argument carry the weight we initially assigned it. Against a properly warm-started probing baseline the advantage is $1.3\text{--}1.7\times$ at the depths where prediction matters on our instances, and probing is sound where the network is not; the ratio reaches $15\text{--}18\times$ only at $n \geq 100$, a regime in which search fails for every strategy we tried. An earlier version of this comparison, using a backend that rebuilt the LP for each probe and excluded the network’s own feature solve from its timer, reported $19\text{--}23\times$ at a scale where the true figure is near unity, and we record the correction because the error came entirely from instrumentation rather than from what the two methods compute.

What remains, and what we claim, is methodological. For planted-solution instance families the exact conditional posterior is computable and is a well-posed supervision signal for branching, where the conventional target — a single planted solution — is not. States drawn from inside the search tree are a better training distribution than root states, and the reduction identity makes generating them a matter of substitution rather than solver instrumentation. The resulting heuristic transfers across instance size when solution multiplicity is held comparable. Whether this becomes practically useful depends on two things we have not done: extending exact supervision past $n \approx 60$, most plausibly by approximating the marginals without exhaustive enumeration, and implementing the search well enough that a branching improvement is visible at the system level.

Reproducibility

Code is organized by component under `proposed_src/`, with shared modules in `util/` and a `ROLES.txt` in each directory describing every file. Scripts are run from their own directory.

```
# generate instances with matched |S| and exact marginals
cd proposed_src/v15_bayes
python3 v15_gen_by_m.py --m 10 --n 25 --count 2200 --K 20 \
    --out ../../runs/data/sols_10x25.json

# build in-tree training states (depths 0-12, 6 per instance)
cd ../v17_conditional
```

```
python3 v17_make_conditional_data.py --sols ../../runs/data/full_10x25_train.json \
--max_depth 12 --per_instance 6 --out ../../runs/data/cond_10x25_train.json

# train (about 20 minutes, single CPU core)
python3 v17_train_conditional.py \
--train ../../runs/data/cond_10x25_train.json \
--test  ../../runs/data/cond_10x25_test.json \
--n_train 10000 --n_test 2000 --epochs 20 --target soft \
--out ../../runs/model_n25

# evaluate: branching arms x {lattice off, lattice on}
cd ../v22_transfer
python3 v22_cascade_search.py --data_dir ../../instances/v22test_21x60 \
--tag EVAL --time_limit 600 --block 12 --tries 10 \
--arms lp model --ckpt ../../runs/model_n25/conditional.pt \
--ahl off on --jobs 6 --out_root ../../runs/eval
```

All generators are seeded. The released checkpoints are 96 KB each. Instance data and weights are regenerated from the above rather than distributed, and the full evaluation cycle is scripted in `v22_transfer/run_v22_cd.sh`.

References

- Aardal, K., Hurkens, C. A. J., and Lenstra, A. K. (2000). Solving a system of linear Diophantine equations with lower and upper bounds on the variables. *Mathematics of Operations Research*, 25(3):427–442.
- Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K., and Smeltink, J. W. (2000). Market split and basis reduction: Towards a solution of the Cornuéjols–Dawande instances. *INFORMS Journal on Computing*, 12(3):192–202.
- Amos, B. (2023). Tutorial on amortized optimization. *Foundations and Trends in Machine Learning*, 16(5):592–732.
- Bengio, Y., Lodi, A., and Prouvost, A. (2021). Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421.
- Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. (2023). On representing mixed-integer linear programs by graph neural networks. In *International Conference on Learning Representations*.
- Cornuéjols, G. and Dawande, M. (1999). A class of hard small 0-1 programs. In *Integer Programming and Combinatorial Optimization*, pages 284–293.
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems*.
- Gershman, S. J. and Goodman, N. D. (2014). Amortized inference in probabilistic reasoning. In *Proceedings of the 36th Annual Meeting of the Cognitive Science Society*, pages 517–522.

- Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B. (2016). Learning to branch in mixed integer programming. In *AAAI Conference on Artificial Intelligence*, pages 724–731.
- Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534.
- Li, Z., Guo, J., and Si, X. (2023). G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. *Transactions on Machine Learning Research*.
- Marques-Silva, J. P. and Sakallah, K. A. (1999). GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521.
- Millidge, B. (2022). Deconfusing direct vs amortized optimization. Unrefereed technical note, <https://www.beren.io/2022-09-25-Deconfusing-direct-vs-amortized-optimization/>.
- Moskewicz, M. W., Madigan, C. F., Zhao, Y., Zhang, L., and Malik, S. (2001). Chaff: Engineering an efficient SAT solver. In *Design Automation Conference*, pages 530–535.
- Ohrimenko, O., Stuckey, P. J., and Codish, M. (2009). Propagation via lazy clause generation. *Constraints*, 14(3):357–391.
- Schnorr, C. P. and Euchner, M. (1994). Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66(1–3):181–199.
- Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill, D. L. (2019). Learning a SAT solver from single-bit supervision. In *International Conference on Learning Representations*.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph attention networks. In *International Conference on Learning Representations*.
- Xu, K., Hu, W., Leskovec, J., and Jegelka, S. (2019). How powerful are graph neural networks? In *International Conference on Learning Representations*.